



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

Titulo TFG



Presentado por Lydia Blanco Ruiz
en Universidad de Burgos — 22 de septiembre
de 2025

Tutor: Nombre Tutor
y Nombre Co-tutor



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Nombre Tutor y Nombre Co-tutor, profesores del departamento de Ingeniería Informática, área de nombre área.

Expone:

Que el alumno D. Lydia Blanco Ruiz, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Título TFG.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 22 de septiembre de 2025

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Nombre Tutor

D. Nombre Co-tutor

Resumen

En este primer apartado se hace una **breve** presentación del tema que se aborda en el proyecto.

Descriptores

Palabras separadas por comas que identifiquen el contenido del proyecto Ej: servidor web, buscador de vuelos, android ...

Abstract

A **brief** presentation of the topic addressed in the project.

Keywords

keywords separated by commas.

Índice general

Índice general	iii
Índice de figuras	iv
Índice de tablas	v
1. Introducción	1
2. Objetivos del proyecto	3
2.1. Objetivos específicos	3
2.2. Objetivos personales	4
3. Conceptos teóricos	5
3.1. LLM	5
3.2. RAG	6
3.3. Conexión LLM con RAG	7
4. Técnicas y herramientas	9
4.1. Metodologías	9
4.2. Repositorio	9
5. Aspectos relevantes del desarrollo del proyecto	11
6. Trabajos relacionados	13
7. Conclusiones y Líneas de trabajo futuras	15
Bibliografía	17

Índice de figuras

Índice de tablas

1. Introducción

Descripción del contenido del trabajo y del estructura de la memoria y del resto de materiales entregados.

2. Objetivos del proyecto

El objetivo principal del proyecto es realizar un sistema de Retrieval-Augmented Generation (RAG) que permita enriquecer las respuestas de un modelo de lenguaje mediante la recuperación y utilización de información contextual proveniente de una colección documental.

2.1. Objetivos específicos

- Preprocesar la colección documental.
 - Segmentar los documentos en fragmentos.
 - Normalizar y limpiar el texto.
 - Generar embeddings para representar semánticamente los fragmentos.
- Implementar y gestionar un índice vectorial.
 - Almacenar los embeddings en una base de datos vectorial.
 - Optimizar consultas de similitud para recuperar contexto relevante.
- Desarrollar la capa de interacción con el usuario
 - Transformar la consulta en su representación vectorial.
 - Recuperar los fragmentos más relevantes del índice.
 - Integrar los fragmentos en el prompt para enriquecer la respuesta del LLM.

2.2. Objetivos personales

- Adquirir conocimientos sobre los sistemas RAG, comprendiendo sus fundamentos, componentes y aplicaciones en el ámbito de la inteligencia artificial.
- Profundizar en el estudio de los modelos de lenguaje de gran tamaño (LLM).
- Desarrollar habilidades prácticas en la implementación de sistemas basados en RAG y LLM.
- Aprender a aplicar los sistemas RAG y los LLM en la resolución de problemas reales.
- Aplicar los conocimientos adquiridos durante la carrera.

3. Conceptos teóricos

En este apartado considero importante definir que es un RAG y cuales son sus aportaciones más significativas a los LLM que ya existen, pero, también, considero importante contextualizar su origen, y para ello voy a empezar comentando que son los LLM y sus principales limitaciones.

3.1. LLM

Los Large Language Model (LLM) son los grandes modelos del lenguaje. Es decir, son modelos de aprendizaje automático que han sido entrenados con conjuntos de datos generados por humanos. Estos datos los usan para encontrar patrones estadísticos y replicar nuestras habilidades lingüísticas. Por lo cual podemos decir que en esencia son modelos de predicción del siguiente token o lo que es lo mismo de la siguiente palabra.

Algunos ejemplos de LLM son ChatGPT, Claude o Llama.

Características

- Naturaleza de predicción del próximo token: como he mencionado anteriormente los LLM seleccionan la palabra más probable de una distribución. La elección de esta palabra se basa en los patrones estadísticos que ha aprendido con los datos del entrenamiento.

- Memoria paramétrica: es el conocimiento interno del LLM. Consiste en la información con la que ha sido entrenado y se basa únicamente en sus parámetros. Por lo cual es una memoria limitada y estática.

Limitaciones

- Desactualizado: los eventos posteriores al entrenamiento del LLM, es decir, la fecha de corte del cocimiento, no estarán disponibles para el modelo lo que puede hacer que la respuesta este desactualizada. Además, el entrenamiento de un LLM es muy costoso y lleva mucho tiempo, por lo cual no se reentrenan con mucha frecuencia.

- Alucinaciones: a veces los LLM dan respuestas incorrectas con gran confianza de manera que parece que la información que te está dando es verdadera. Esto se debe a su naturaleza de predicción del próximo token.

- Limitación de conocimiento: los LLM se entrenan con datos públicos esto limita su conocimiento, ya que no tienen conocimientos de información que no sea publica, como, por ejemplo, documentos internos de una empresa, información de los clientes o de los productos.

Solución

Para solventar las limitaciones que hemos comentado la solución es darle al LLM el contexto que le falta. Para proporcionarle este contexto podemos expandir la memoria del LLM mediante generación aumentada, es decir, incorporando un RAG.

3.2. RAG

Retrieval Augmented Generation (RAG), o en español, generación aumentada por recuperación es la técnica que consiste en recuperar la información que sea relevante de una fuente externa, con dicha información aumenta la entrada al LLM, lo que permite que el LLM genere una respuesta mucho más precisa, contextual y confiable. Para realizar esto el RAG usa tres pasos, recuperar, aumentar y generar.

EL RAG combina dos tipos de memoria:

- La memoria paramétrica: es la que épicamente tienen los LLM, que como hemos visto es limitada y estática.

- La memoria no paramétrica: es información externa al LLM que se le puede proporcionar. Es un conocimiento externo y dinámico que se puede extender tanto como queramos y que puede contener documentos propietarios.

Por lo cual conociendo esto, podemos decir que la generación aumentada por recuperación (RAG) es una metodología que busca ampliar la memoria paramétrica de un LLM incorporando una memoria no paramétrica explícita. A través de un recuperador, se obtiene información relevante de esta memoria externa, la cual se añade al prompt y luego se envía al LLM. De esta forma, el modelo puede producir respuestas más contextuales, confiables y precisas.

Objetivos del RAG

- Hacer que los LLM respondan con información actualizada.
- Hacer que los LLM respondan información precisa, contextual y confiable.
- Hacer que los LLM conozcan información propietaria.

Ventajas

- Conocimiento del contexto profundo: esto se debe a que la información adicional ayuda al LLM a generar respuestas contextualmente apropiadas, ya que, las respuestas se basan en datos específicos. Esto aumenta la confianza del usuario.
- Citar fuentes: como la información se extrae de fuentes conocidas las puede citar en su respuesta. Esto dispara la fiabilidad y permite a los usuarios comprobar la información obtenida.
- Reduce las alucinaciones: el sistema esta anclado a los hechos, por lo que se reducen las respuestas inexactas o falsas.

3.3. Conexión LLM con RAG

Cogemos los documentos externos y los convertimos en vectores numéricos (embeddings) y los almacenamos. Cuando llega una consulta, en lugar de buscar coincidencias por palabras buscamos en ese espacio vectorial los trozos que son semánticamente más parecidos, a esto se le conoce como búsqueda por significado. Esos fragmentos se incorporan como contexto al LLM, de modo que RAG combina la capacidad de razonamiento del modelo con una recuperación de información altamente precisa.

4. Técnicas y herramientas

4.1. Metodologías

Scrum

La metodología Scrum es un marco de trabajo ágil orientado a la gestión y desarrollo de proyectos de manera iterativa e incremental. Se organiza en ciclos cortos denominados sprints, al final de los cuales se entrega un incremento funcional del producto. Estos sprints favorecen la mejora continua y la adaptación al cambio, además, aseguran una entrega temprana de valor. A diferencia de metodologías tradicionales, Scrum aporta mayor flexibilidad y retroalimentación constante [2].

4.2. Repositorio

Para alojar el repositorio he valorado distintas alternativas como Bitbucket, GitHub, GitLab, Trello, XpDev o PivotalTracker. Finalmente he elegido GitHub que es una plataforma basada en la nube que permite alojar repositorios de código. GitHub ofrece funciones muy útiles como ramas (branches), pull request, seguimiento de cambios (commits), historial del proyecto, gestión de incidencias (issues). Además, permite la colaboración entre usuarios, lo que es muy útil para la revisión del proyecto.

Otra ventaja es que GitHub se integra con herramientas de integración y despliegue continuo, lo que facilita la automatización de pruebas y la entrega de software. Asimismo, permite incluir documentación directamente en el repositorio, ya sea mediante archivos README o a través de Wikis, lo que mejora la claridad y la reproducibilidad del trabajo. Finalmente, al tratarse

de la plataforma más utilizada a nivel mundial, cuenta con abundante documentación, lo que la convierte en una alternativa sólida y con amplio soporte [1].

5. Aspectos relevantes del desarrollo del proyecto

Este apartado pretende recoger los aspectos más interesantes del desarrollo del proyecto, comentados por los autores del mismo. Debe incluir desde la exposición del ciclo de vida utilizado, hasta los detalles de mayor relevancia de las fases de análisis, diseño e implementación. Se busca que no sea una mera operación de copiar y pegar diagramas y extractos del código fuente, sino que realmente se justifiquen los caminos de solución que se han tomado, especialmente aquellos que no sean triviales. Puede ser el lugar más adecuado para documentar los aspectos más interesantes del diseño y de la implementación, con un mayor hincapié en aspectos tales como el tipo de arquitectura elegido, los índices de las tablas de la base de datos, normalización y desnormalización, distribución en ficheros³, reglas de negocio dentro de las bases de datos (EDVHV GH GDWRV DFWLYDV), aspectos de desarrollo relacionados con el WWW... Este apartado, debe convertirse en el resumen de la experiencia práctica del proyecto, y por sí mismo justifica que la memoria se convierta en un documento útil, fuente de referencia para los autores, los tutores y futuros alumnos.

6. Trabajos relacionados

Este apartado sería parecido a un estado del arte de una tesis o tesina. En un trabajo final grado no parece obligada su presencia, aunque se puede dejar a juicio del tutor el incluir un pequeño resumen comentado de los trabajos y proyectos ya realizados en el campo del proyecto en curso.

7. Conclusiones y Líneas de trabajo futuras

Todo proyecto debe incluir las conclusiones que se derivan de su desarrollo. Éstas pueden ser de diferente índole, dependiendo de la tipología del proyecto, pero normalmente van a estar presentes un conjunto de conclusiones relacionadas con los resultados del proyecto y un conjunto de conclusiones técnicas. Además, resulta muy útil realizar un informe crítico indicando cómo se puede mejorar el proyecto, o cómo se puede continuar trabajando en la línea del proyecto realizado.

Bibliografía

- [1] GitHub Inc. Acerca de github y git. <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>, 2025. [Online; Accedido 22-Septiembre-2025].
- [2] Jeff Sutherland Ken Schwaber. The scrum guide. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>, 2020. [Online; Accedido 22-Septiembre-2025].